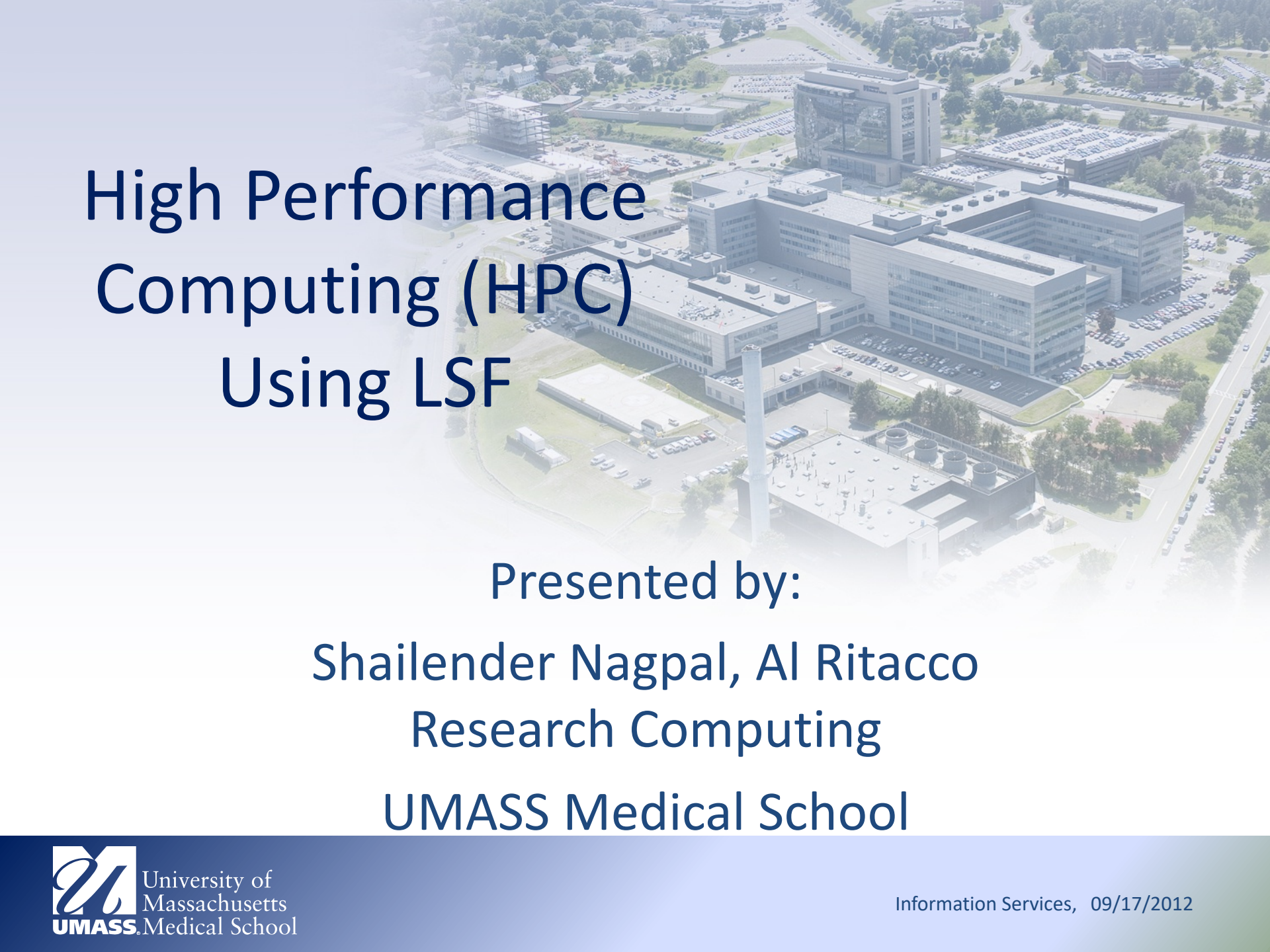




High Performance Computing (HPC) Using LSF

Presented by:
Shailender Nagpal, Al Ritacco
Research Computing
UMASS Medical School



University of
Massachusetts
UMASS Medical School

Agenda

- Introduction to HPC
- UMASSRC Resources
- Resource management
- Submitting a job
- Using the right queue(s)
- Creating shell scripts for jobs
- Q&A

What is HPC?

- HPC = High Performance Computing
 - Computing infrastructure in which hundreds or thousands of computers are networked, share storage and software
 - Environment can be leveraged together to complete “big” computing tasks
 - Multiple users can login and use the infrastructure
 - More than 1 computer can be used to complete a computing task
 - Special tools/skills required to leverage HPC environment – Linux, LSF commands (or another Scheduler)

Definitions

HPC Term	Definition
Node	A single computer available to perform computing tasks
Rack	A cabinet in which multiple nodes can be stacked vertically and/or horizontally, allowing for efficient housing, networking and power management
Cluster	A collection of computer “nodes” that are on the same network for inter-node communication, shared storage and to execute jobs
CPU	A CPU is the electronic circuitry (Microprocessor) within a computer that carries out the instructions of a computer program
Core	Independent programming unit within a CPU that can execute program instructions. A modern CPU can have multiple cores
Head node	In a cluster, one or a few nodes can be designated as a head node where users typically are able to login and create/monitor jobs

Definitions (...contd)

HPC Term	Definition
Compute node	Compute nodes in a cluster execute a job created by a head node. Users cannot login into a compute node
Process	A process is an instance of a computer program that is being executed. It contains the program code and its current activity
Thread	A thread is the smallest sequence of programmed instructions that can be managed independently by the scheduler of an OS
Job	A job is a linux command that is designated to be executed on a compute node rather than the head node
Job array	Identical jobs that have a different iterator variable
Parallel job	Jobs that break a complex computing task into smaller tasks, such that each task is executed on different nodes simultaneously
Queue	Designated “lanes” for submitting different types of jobs depending on priority, resources required or expected duration of execution

Definitions (...contd)

HPC Term	Definition
Scheduler	HPC software that allows for efficient utilization of cluster resources based on submitted job types
Job Management	HPC software that keeps track of jobs submitted
Research computing	One of the departments within Umass Medical School responsible for supporting the HPC infrastructure on campus. Not related to “IT”
Cloud computing	A variant of HPC infrastructure which is not limited to a particular organization, where computing resources are requested on demand
Distributed computing	Buzzword similar to High Performance Computing
Parallel computing	Buzzword similar to High Performance Computing

Why do you need HPC?

- Needs assessment:
 - Use software that's only available on linux
 - Install it yourself on your own linux PC?
 - RC already has it installed?
 - Automate data crunching tasks
 - Routine incoming data that needs to be crunched?
 - Workflow available within RC to handle it?
 - Simulations
 - Molecular dynamics simulations taking too much time?

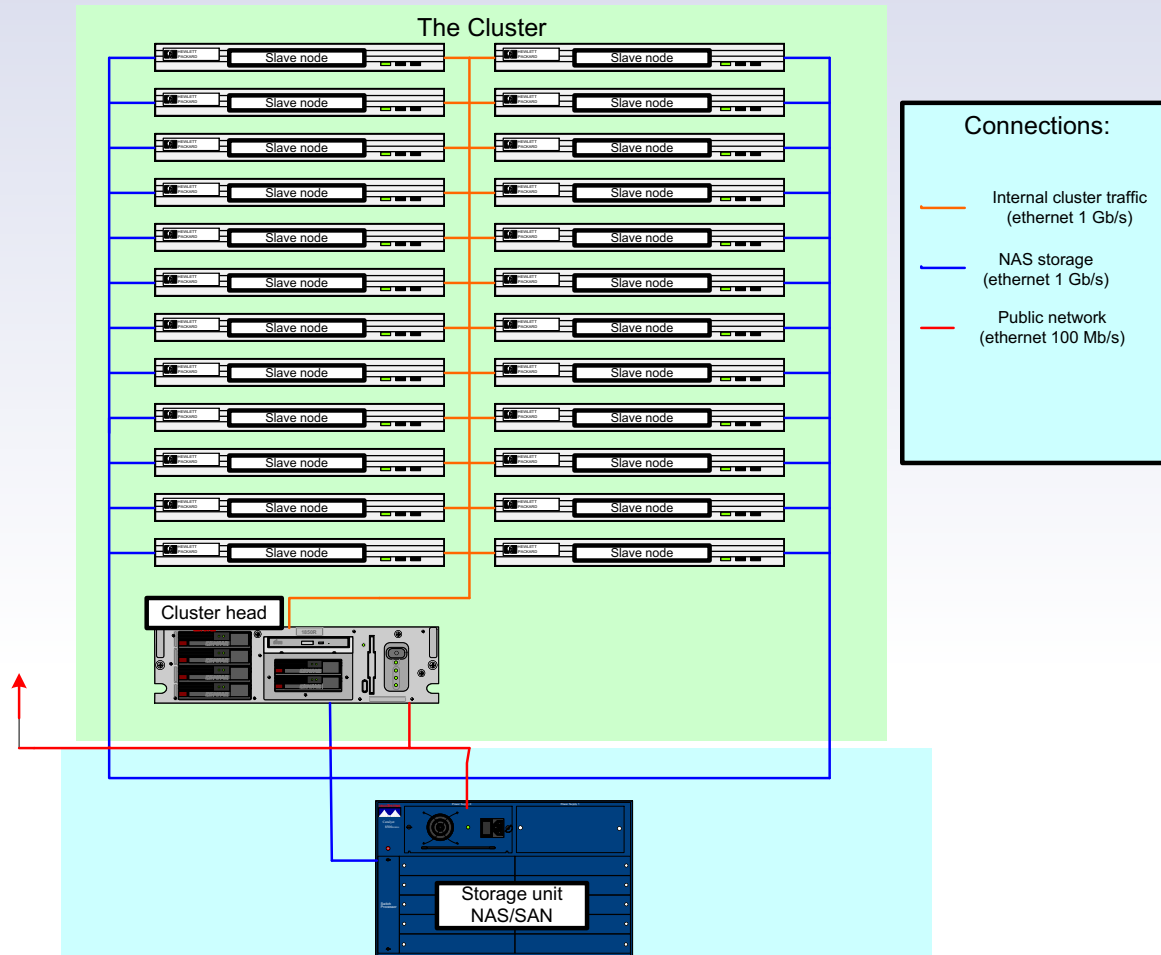
HPC is not for these!

- To run windows software with point-n-click interfaces
- Working with office documents – spreadsheets, slides, etc
- Video games, music or general video
- Web browsing
- Emails

Policies for HPC use

- If you have a “need” to use HPC, RC group can help, but there are expectations:
 - Understanding of the constraints of our HPC implementation – CPUs, memory, local and shared storage, networking, etc
 - Good knowledge of your own tasks/jobs that you are going to run – expected run times, utilization of memory, disk space and network bandwidth
 - Fair share policies

Typical HPC environment



HPC infrastructure at UMass RC

- Massachusetts Green High Performance Computing Cluster
 - 10264 cores available, each node has 196 - 512 GB RAM.
12 GPU nodes available
 - 400TBs of high performance EMC Isilon X series storage
 - FDR based Infiniband (IB) network and a 10GE network for the storage environment
- Software related to research installed:
 - Physics, Medical Physics, Genomics, Chemistry...

What does my Account provide?

- 50 GB HOME Directory (Backed up)
 - Used to store local files used by you
- Project based storage is available per PI
 - Sizes range from 100GB – several TBs
 - Used to store files for your lab
 - A PI can request storage for their lab to access
- Software:
 - We have pre-loaded software using modules
 - List of software can be shown via: \$ modules avail

How do I get help?

- We are available M-F 9-5 and can be emailed via:
HPCC-Admins <hpcc-support@umassmed.edu>
- Can I install software?
 - Yes
- Will the HPC team help me install software?
 - Yes, for most packages. It must be an HPC package
- Can you help me with running my program
 - Yes, please contact us

Processes, Tasks, Cores, CPUs

- Processes and tasks
 - Single process OS and processes
 - Single processing OSs can run only one user process at a given time, a single task
 - All tasks run until completion before another task is started (MS-DOS)
 - Concurrent (Multi-tasking) OS and processes
 - A concurrent OS provides users the ability to execute many programs simultaneously
 - Linux provides users the ability to execute: an editor, a music player, and other tasks simultaneously, thus allowing for multi-tasking
 - Multiple cores (SMP) and Linux processes
 - A process which can take advantage more than one core while running. These are typically called: multi-threaded.

Short Review

- If a node has four CPUs and two cores per CPU, how many total cores are there?
- In Linux can we execute an editor and a program to search a genome at the same time?
- How many processes should we execute on a node which has two CPUs with 8 cores each? (one core, one process is ideal!)

What is a computing “Job”?

- A computing “job” is an instruction to the HPC system to execute a command or script
 - Simple linux commands that can be executed within milliseconds would probably not qualify to be submitted as a “job”
 - Any command that is expected to take up a big portion of CPU or memory for more than a few seconds on a node would qualify to be submitted as a “job”. Why? (Hint: multi-user environment)

How to submit a “job”

- The basic syntax is:

```
bsub <valid linux command>
```

- bsub: LSF command for submitting a job
- Lets say user wants to count number of lines in a FASTQ file. On a linux PC, the command is

```
wc -l reads.fastq
```

- To submit a job to do the work, do

```
bsub wc -l reads.fastq
```

Specifying more “job” options

- Jobs can be marked with options for better job tracking and resource management
 - Job should be submitted with parameters such as queue name, estimated runtime, job name, memory required, output and error files, etc.
- These can be passed on in the bsub command

```
bsub -q short -W 1:00 -R rusage[mem=2048] -J  
"Myjob" -o hpc.out -e hpc.err wc -l reads.fq
```

Common Job submission “options”

Option flag or name	Description
-q	Name of queue to use. On our systems, possible values are “short” (<=4 hrs execution time), “long” and “interactive”
-W	Allocation of node time. Specify hours and minutes as HH:MM
-J	Job name. Eg “Myjob”
-o	Output file. Eg. “hpc.out”
-e	Error file. Eg. “hpc.err”
-R	Resources requested from assigned node. Eg: “-R rusage[mem=1024]”, “-R hosts[span=1]”
-n	Number of cores to use on assigned node. Eg. “-n 8”

Submitting scripts to LSF

- You can include bsub options in a shell script at the top of the file
- Prefix each line with #BSUB
- One per line
- Script will run as normal without LSF so you can test
- Rest of script is a normal shell script

Creating shells scripts for jobs

Create a script “hello-job.sh”

```
#!/bin/bash

#BSUB -q short
#BSUB -W 00:10
#BSUB -n 1
#BSUB -R "rusage [mem=1024] "
#BSUB -J "myJob"
#BSUB -o lsf.out
#BSUB -e lsf.err
echo "Hello Job $LSB_JOBID. I was run on `hostname`"
```

To execute on shell, run: `bsub < hello-job.sh`

LSF based array of jobs

- Say you have 1000 files that you need to process and they're numbered sequentially file 1 through file 1000
- `bsub -W 1:0 -R "rusage[mem=1024]" \`
`-J "myarray[1-1000]" \`
`"process file.\$LSB_JOBINDEX"`
- This will run jobs 1..1000, one slot per

Creating array of jobs

Create a script “hello-job-array.sh”

```
#!/bin/bash

#BSUB -q short
#BSUB -W 00:10
#BSUB -n 1
#BSUB -R "rusage[mem=1024]"
#BSUB -J "myTask[1-10]"
#BSUB -o logs/out.%J.%I

echo "Hello Job $LSB_JOBID Task $LSB_JOBINDEX was
run on `hostname`"
```

To execute on shell, run: `bsub < hello-job-array.sh`

Passing options to shell scripts

- This works:

```
bsub myjob.sh opt1 opt2 opt3
```

- This won't:

```
bsub < myjob.sh opt1 opt2 opt3
```

- Write wrapper for bsub that passes arguments

```
#!/bin/bash
```

```
bsub -q short myjob.sh $1 $2 $3
```

- Then run

```
./mywrapper.sh opt1 opt2 opt3
```


Special LSF variables for use in scripts

- LSF provides a few shell environment variables that can be used within scripts

<code>LSB_ERRORFILE</code>	Name of the error file
<code>LSB_JOBID</code>	Batch job ID assigned by LSF
<code>LSB_JOBINDEX</code>	Index of the job that belongs to a job array
<code>LSB_QUEUE</code>	Name of the queue the job is dispatched from
<code>LSB_JOBNAME</code>	Name of the job
<code>LSB_SUBCWD</code>	The directory where the job was submitted
<code>LSB_HOSTS</code>	The list of hosts that are used to run the batch job

- Try out the script:

```
bsub < env-variables-job.sh
```

Learning to use HPC

- Linux is a pre-requisite to using any HPC system
 - Plenty of linux tutorials on the internet
 - Attend our “Intro to linux” sessions when offered
- Our website is a good resource for learning to use HPC, visit
www.umassrc.org
- Lots of examples provided

Bioinformatics HPC demo

- Log on to the Umass server using Putty on windows or Terminal on Mac
- Request an interactive shell session on one of the compute nodes for this demo

```
$ bsub -q interactive -W 4:00 -Is bash
```

- Navigate to the training directory or copy the examples to your local directory

```
$ cd /project/umw_rcs/training/hpc
```

OR

```
$ cp /project/umw_rcs/training/hpc/* .
```



Bioinformatics HPC demo (...contd)

- Create a bowtie script, “bowtie-script.sh”

```
#!/bin/bash
# Create variables for files to be used
ref="ref.fasta"; reads="reads.fq"

# Load modules
module load bowtie2/2-2.1.0; module load samtools/1.2

# Build reference sequence index
bowtie2-build $ref ref

# Perform alignment using multi-threading
bowtie2 -p 8 -x ref $reads -S ${reads}.sam

# Convert to BAM
samtools view -b ${reads}.sam -o ${reads}.bam
```

Bioinformatics HPC demo (...contd)

- Now submit this script as a compute job

```
bsub -W 4:00 -q short -R "rusage[mem=4096]" -R  
"rusage[span=1]" -J "bowtie-job" -n 8 -o ngs.out  
-e ngs.err ./bowtie-script.sh
```

- Another way of writing the script is to include all of the command line options into the script itself (next slide)

Bioinformatics HPC demo (...contd)

```
#!/bin/bash
#BSUB -J "SeqAlignJob"
#BSUB -R rusage[mem=4096]
#BSUB -R rusage[span=1]
#BSUB -q short
#BSUB -W 4:00
#BSUB -n 8
#BSUB -o ngs.out
#BSUB -e ngs.err
./bowtie-script.sh
```

- Then save as “bowtie-job.sh”, submit as

```
bsub < bowtie-job.sh
```



Why manage resources?

- Accurately listing requirements means your job gets dispatched faster
- Does your job scale well?
 - Doubling the core count may not halve run time
- Can it be distributed?

LSF based resource requests

- Non-threaded job Example:
 - A job requiring: 4 cores, a run time of 50 minutes, and 1GB of memory per job slot:
 - `bsub -q short -n 4 -W 0:50 \`
 - `-R "rusage[mem=1024]" \`
 - `-R "span[hosts=1]"`
 - `./myparalleljob.sh`
- Here we run in the short queue as we are looking for singleton based jobs

Resource Requests, cont.

- Threaded jobs
- A job requiring: 4 cores, a run time of 50 minutes, and 1GB of memory per job slot:
- `bsub -q parallel -n 4 -W 0:50 \`
`-R "span[hosts=1]" -R \`
`"rusage[mem=1024]" ./mythreadedjob.sh`
- Here we request the parallel queue as this job although not running on separate hosts are considered parallel in nature

Resource Requests, cont.

- GPU resource requests
 - Single GPU:
 - `#BSUB -q gpu -a gpuexcl_p`
 - Both GPUs on a host:
 - `#BSUB -q gpu -a gpuexcl2_p`
 - Resource requests for memory, and time are still the same here
 - Note, these hosts are X86 – Intel and not AMD nodes
 - If you are using CUDA don't forget to load the module

Resource Requests

- Submitting a job with basic resources:
- Memory
 - Using the -R option
- Request ~16GB Ram

```
$ bsub -R "rusage[mem=16000]" script.sh
```

Resource Requests, cont.

- How much time does your job need?
 - Estimate the amount of time at first
 - If you know it takes 1 hour for 1 CHR, and you are analyzing 4 CHR, then figure 5 hours and see where things end up
- Used the –W H:M resource request:
 - #BSUB -W 5:00

Resource Requests, cont.

- Which queue do we want to run in?
 - Recall the different types of queues and select
 - Default is long
- Using `-q {queue name}` we select queue
- `#BSUB -q short`

Resource Requests, cont.

- How do I run a threaded application?
 - All cores/slots need to be on the same host
 - All memory is in the same name space
- Use the -R “span” resource request to make sure that you have ALL cores on ONE host as:
 - #BSUB -R "span[hosts=1]"

Resource Requests, Cont.

- How do I run an MPI job?
 - Cores/slots can reside on any host
 - Optimal setup is to run on same host as much as possible of course
 - Namespace is independent
 - Load the MPI module
- -R “span” with ptile
- Example: 12 slots with 6 slots for each host
 - #BSUB -n 12
 - #BSUB -R "span[ptile=6]"

Ex: LSF Submission with resources

Say we have a need for 15GB of available RAM, and 4 slots/CPU's for our job to run. We still use the `bsub` command but with options:

- `-R rusage[mem=X]` – Request X MB of RAM
- `-n X` – Request X Slots (cores)
 - Job slots are not guaranteed to be on the same system unless we tell LSF for them to be

```
$ bsub -R rusage[mem=15360] -n 4 ./vmd.sh
```


Measuring resources used

- Job report contains memory and CPU used
- Also contains the amount of time the job was pending and running
- GHPCC optimized for short (<12H) jobs
- Test software with larger requirements and use that as a gauge
- Run production with slightly scaled up numbers

Looking up submitted jobs

- Upon submission, we get the job id
Job <3483897> is submitted to queue <short>
- We can also look up a job by entering the “bjobs” command

```
bjobs
```

JOBID	USER	STAT	QUEUE	FROM_HOST	EXEC_HOST	JOB_NAME	SUBMIT_TIME
3483897	sn34w	PEND	short	ghpcc06		myJob	Jun 15 11:35

```
bjobs -a # Show all jobs
```

JOBID	USER	STAT	QUEUE	FROM_HOST	EXEC_HOST	JOB_NAME	SUBMIT_TIME
3483893	sn34w	DONE	short	ghpcc06	ghpcc-sgi	-o	Jun 15 11:29
3483896	sn34w	DONE	long	ghpcc06	c11b02	myJob	Jun 15 11:33
3483897	sn34w	DONE	short	ghpcc06	ghpcc-sgi	myJob	Jun 15 11:35

Killing a running job

- **BKILL – Stops a job (or send a specific signal)**

```
bkill {jobid}           # stop a specific jobid
```

```
bkill 0                 # stops all your jobs
```

```
bkill -q long 0         # stop all jobs in long queue
```

Peeking at a running job

- BPEEK – view output (stderr/out) running job
bpeek 4587

Suspending Jobs

- **BSTOP** – put a job in suspend state

```
bstop 174224
```

```
# put job on hold
```

JOBID	USER	STAT	QUEUE
174224	awr	USUP	long

FROM_HOST	EXEC_HOST	JOB_NAME	SUBMIT_TIME
ghpcc06	c06b01	*;sleep 30	May 13 11:14

Resume a suspended Job

- BRESUME JOBID – Resume job

```
bresume 174224
```

JOBID	USER	STAT	QUEUE	FROM_HOST	EXEC_HOST	JOB_NAME	SUBMIT_TIME
174224	awr	RUN	long	ghpcc06	c06b01	*;sleep 30	May 13 11:14

Which queues are available?

- BQUEUES lists available queue resources

bqueues

QUEUE_NAME		PRIO	STATUS	MAX	JL/U	JL/P	JL/H	NJOBS	PEND	RUN	SUSP
priority	100	Open:Active	- - - -	0	0	0	0				
interactive	90	Open:Active	- 5 - -	3	0	3	0				
short	80	Open:Active	2854 - - -	1378	628	750	0				
parallel	70	Open:Active	2854 - - -	6049	3195	2721	0				
long	60	Open:Active	- - - -	1268	372	708	112				
gpu	50	Open:Active	- - - -	0	0	0	0				
condo_uma_haiyi	30	Open:Active	- - - -	6912	6432	480	0				
condo_grid	20	Open:Active	- - - -	0	0	0	0				

Why use the correct queue?

- Match requirements to resources
- Jobs dispatch quicker
- Better for entire cluster
- Help GHPCC staff determine when new resources are needed

LSF Hosts

- BHOSTS – show hosts and their status

```
bhosts
```

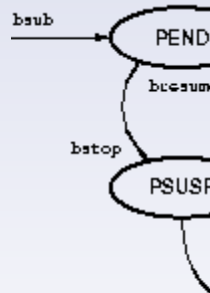
HOST_NAME	STATUS	JL/U	MAX	NJOBS	RUN	SSUSP	USUSP	RSV
c10b08	closed	-	64	64	64	0	0	0
ghpcc-sgi	ok	-	512	489	489	0	0	0

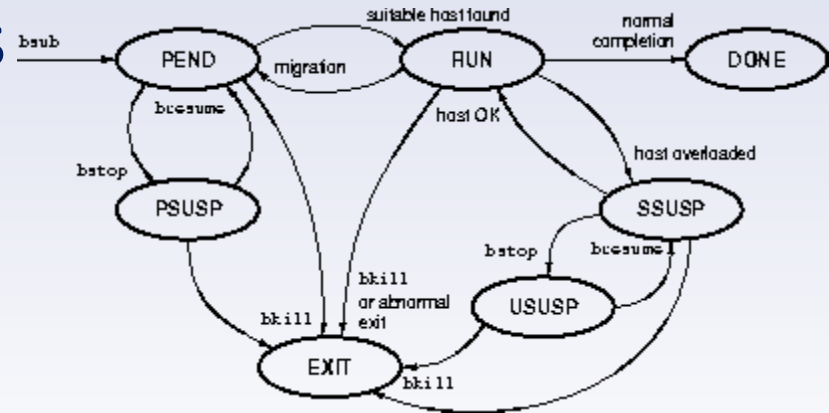
- Status – Shows if host is accepting new jobs
- JL/U – Max Job slots available (per user) ALL
- MAX – Max jobs slots per host
- NJOBS – Jobs running on this host (slots used)

How does LSF schedule job?

- LSF selects which job to run next based on:
 - Resources requirements of the applications
 - Queue
 - Memory
 - Time
 - Job requirements
 - Current load conditions
 - How many slots are available to backfill

Job State(s)

- State Diagram of LSF jobs
 - PEND
 - waiting in the queue
 - RUN
 - dispatched to a host and running
 - DONE
 - terminated normally
- 
- ```
graph TD; subgraph "State Diagram of LSF jobs"; PEND([PEND]); PSUSP([PSUSP]); PEND -- bsub --> PSUSP; PSUSP -- bstop --> PEND; end;
```



# Job State(s), cont.

- A job remains pending until all conditions for its execution are met. These conditions include:
  - Start time specified by the user when the job is submitted
  - Load conditions on qualified hosts
  - Time windows during which the job's queue can dispatch jobs and qualified hosts accept jobs
  - Job limits imposed by the configured policy for each user, queue, and host
  - Relative priority to other users and jobs
  - Availability of the specified resources

# Job State(s), cont.

- A job may terminate abnormally for various reasons. An abnormally terminated job goes into EXIT state. The situations where a job terminates abnormally include:
  - The job is cancelled by its owner or the LSF administrator while pending, or after being dispatched
  - The job is not able to be dispatched before it reaches its termination deadline and thus is aborted by LSF Batch

# Job State(s), cont.

- The situations where a job terminates abnormally include:
  - The job fails to start successfully. For example, the wrong executable is specified by the user when the job is submitted
  - The job crashes during execution

# Parallel versus single

- What is a single job
  - Uses one core/slot and runs in a single namespace
- What is a parallel job
  - Uses more than one core/slot and runs in a single or multiple namespace context
- What is a threaded job
  - A parallel job which runs in a single namespace
- What is an MPI job
  - A parallel job which runs in multiple namespaces

# HPC Best practices

- When submitting a large number of jobs please consider:
  - Single CPU jobs versus multi CPU Jobs
  - Correct amount of memory for your job
  - Job Arrays
  - Job dependencies



# HPC Best practices for disk I/O

- Archive your data
  - Make backups of your data on mid-long term storage
- Use local storage if possible
  - Local storage always faster than network
- Don't use farline for cluster processing

# HPC Best practices (...contd)

- The earlier your jobs are submitted the earlier your job will gain needed LSF resources.
- Re-direct all LSF output to one directory for convenience
- Add the following to your LSF / Job directives: (redirects stdout/stderr)

```
#BSUB -o $HOME/LSF_jobs_output/LSF_job.%J.out
```

```
or
```

```
#BSUB -o $HOME/LSF_jobs_output/LSF_job.%J.%I.out
```

# HPC Best practices for LSF Queues

- Fair share attempts to equalize CPU (slot) resources for Labs and users at job submission.
- The priority of a job is calculated in relation to other submitted jobs. The priority for jobs will change as jobs complete and job slots become available
- All labs start with an equal weight
- Each lab member shares in this weight when submitting jobs
- Weights are measured from job submissions per user and per lab
- Weights are based on CPU time used and a decay time

# Q&A

- Don't be shy – everyone has a different work flow and we want to help you